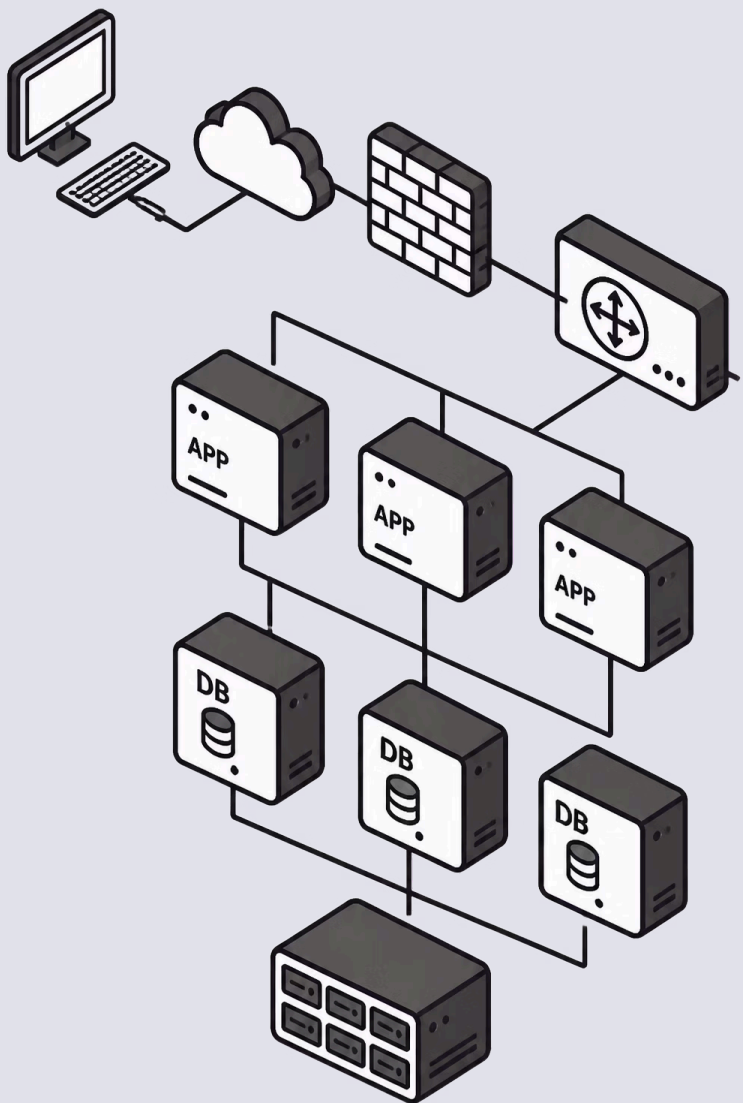
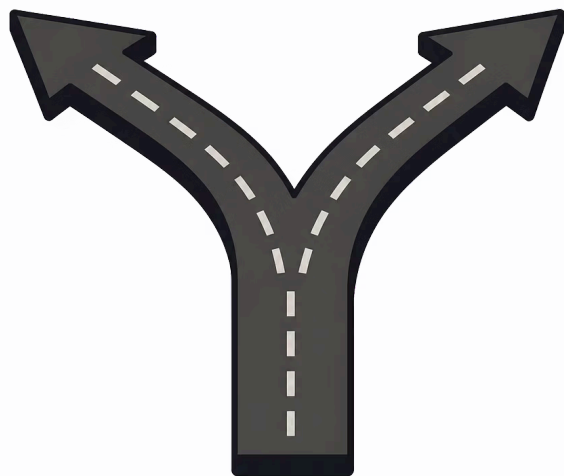




Camunda 7 vs Camunda 8

Что выбрать при разных требованиях к архитектуре, эксплуатации и масштабированию

Короткий ответ: для нового проекта — Camunda 8, однако...



Camunda 8 —
базовый выбор

Для новых
решений с cloud-
native
архитектурой,
высокой
доступностью,
горизонтальным
масштабирование
м и работой в SaaS
или Self-Managed.

Camunda 7 —
проверенный
старичок

Выбор оправдан
прежде всего для
уже работающих
решений, где
важны
минимальные
изменения и
совместимость с
текущим
embedded-стеком.

Официальная
позиция
Camunda

Все новые проекты
— на Camunda 8.
Camunda 7
Enterprise Edition
поддерживается
**до апреля 2030
года.**

Главное различие — где живёт workflow engine

Camunda 7: Embedded Engine

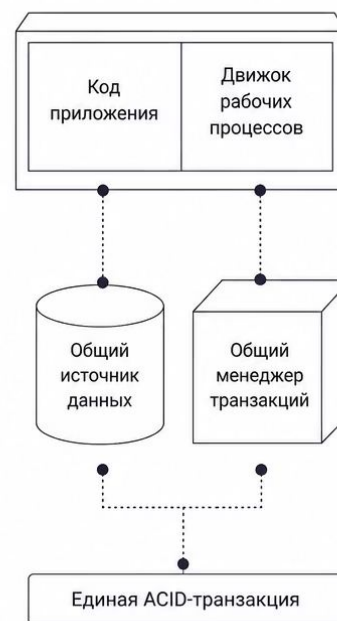
Движок встраивается прямо в приложение — общий JVM, thread pools, datasource и transaction manager. Бизнес-код и workflow участвуют в одной ACID-транзакции.

Camunda 8: Remote Engine (Zeebe)

Zeebe всегда является **удалённым ресурсом**. Embedded engine не поддерживается. Нельзя разделить единую техническую транзакцию между бизнес-кодом и движком.

⚠ Требуется явные паттерны согласованности, идемпотентности и повторов.

Camunda 7
приложение и движок рабочих процессов работают в одном JVM



Camunda 8 (Zeebe)
приложение и движок рабочих процессов — отдельные сервисы, связанные через удаленный API



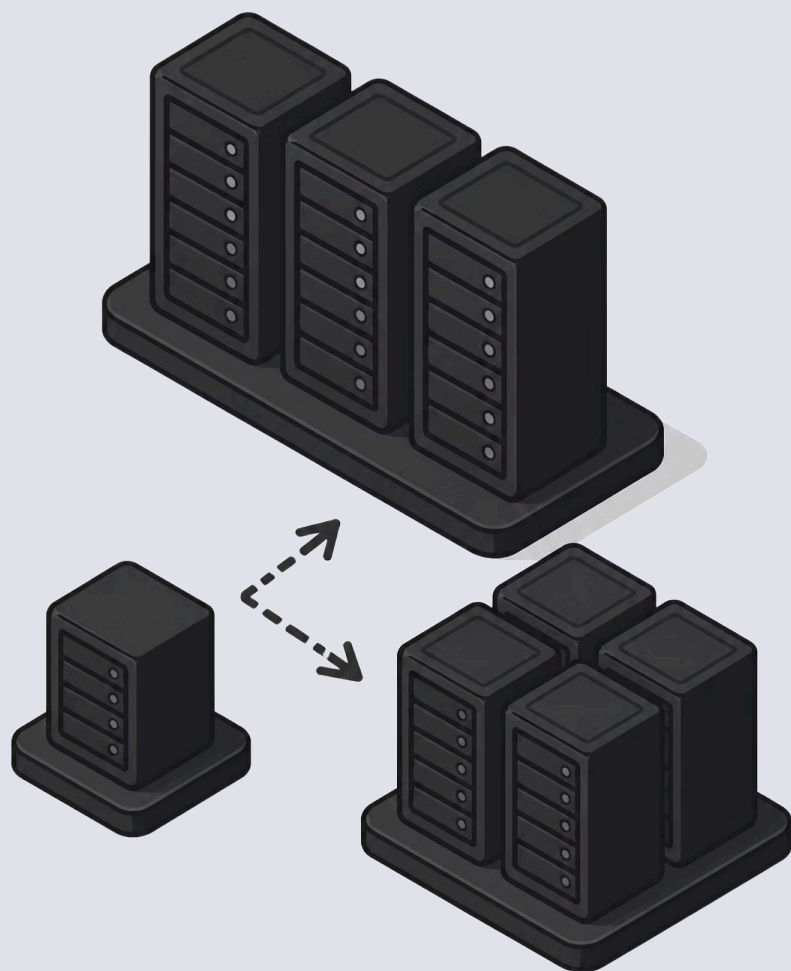
Проблема Camunda 7

Camunda 7: Реляционная СУБД

Традиционная архитектура Camunda 7 опирается на реляционные базы данных (PostgreSQL, Oracle, SQL Server). Это может стать узким местом под высокой нагрузкой, так как каждое изменение состояния workflow требует транзакционной записи в БД, что ограничивает общую пропускную способность.

Camunda 8: Zeebe и Event Sourcing

Zeebe (движок Camunda 8) использует архитектуру на основе Event Sourcing с лог-ориентированным хранилищем (RocksDB). Такой подход обеспечивает горизонтальное масштабирование, высокую доступность и значительно превосходит пропускную способность традиционных СУБД, снимая ограничения по производительности.



Если приоритет — масштабирование и отказоустойчивость

Высокий throughput

Большое количество
параллельных экземпляров
процессов и длинные
процессы без деградации
производительности.

Горизонтальное масштабирование

Независимое
масштабирование workers под
нагрузку. Архитектура
спроектирована для cloud-
native среды.

Гибкость поставки

SaaS или Self-Managed — выбор под ограничения организации.
Централизованный operational monitoring.



Вывод: Если процессы должны выдерживать высокую нагрузку и работать в распределённой среде — выбирайте Camunda 8.



Если нет требований к высокой нагрузке — Camunda 7 остаётся хорошим выбором

Предсказуема я нагрузка

Идеально для
решений с
умеренным
throughput и
предсказуемой
нагрузкой.

Упрощенная архитектура

Embedded engine:
одна JVM, общие
транзакции,
меньше
операционной
сложности.

Низкая задержка

Меньше сетевых
вызовов и latency
благодаря
встроенному
движку.

Простая отладка

Легче отладка и
мониторинг в
едином runtime.

Проверенное решение

Хорошо
зарекомендовала
себя в production
для средних
корпоративных
приложений.

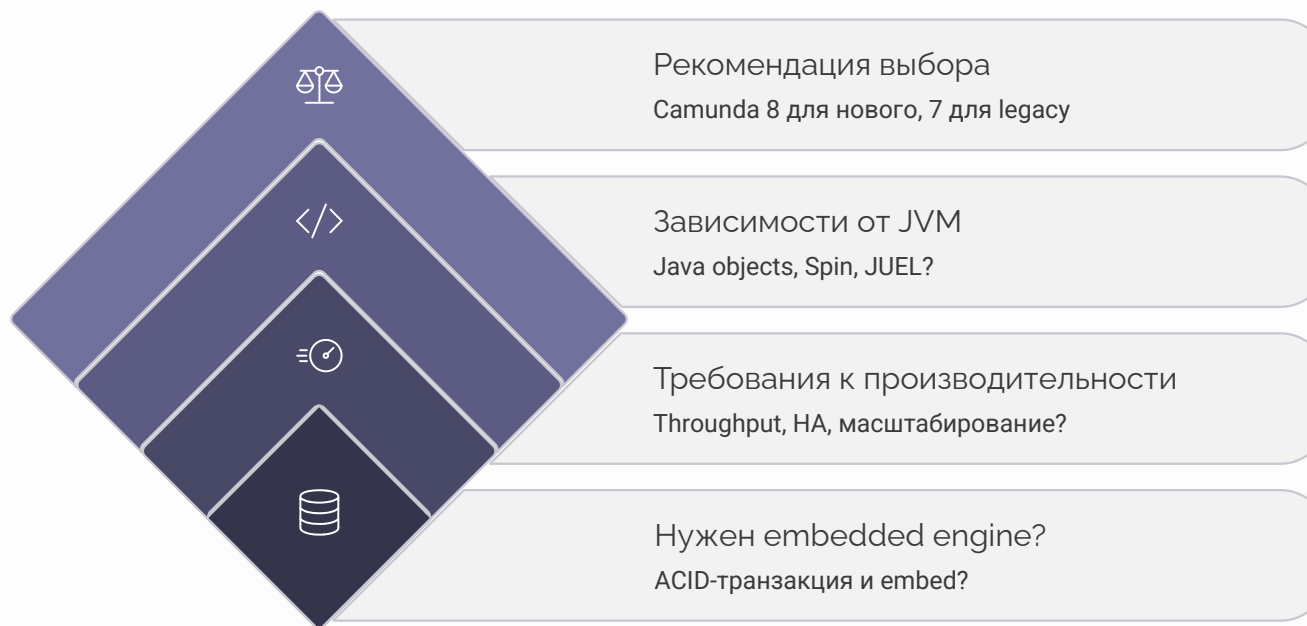
Длительная поддержка

Поддержка до
апреля 2030 года
для планомерной
миграции.

Практическая матрица выбора

Сценарий	Camunda 7	Camunda 8
Новый enterprise-проект	✓ если нет требований к масштабированию	✓
Высокая нагрузка, HA, cloud-native, SaaS/Self-Managed	—	✓
Монолит, shared DB/transactions, Java Delegates	✓ (если обязательно)	—
AI-native capabilities, AI agent orchestration	—	✓
Разработка на любых языках программирования	—	✓

Рекомендация: выбирать профиль требований, а не версию



Последовательный анализ профиля требований позволяет принять обоснованное архитектурное решение, а не следовать инерции.

❏ **Ключевой вопрос:** Какие ограничения Camunda 7 являются бизнес-критичными, а какие лишь привычны команде?

Шаг 1

Нужен ли embedded engine и общая ACID-транзакция с приложением?

Шаг 2

Каковы требования к throughput, HA, масштабированию и модели эксплуатации?

Шаг 3

Есть ли зависимость от Java objects, Spin, JUEL, legacy connectors и multi-tenancy?

Шаг 4

Для нового проекта без ограничений — Camunda 8. Для legacy с обоснованными ограничениями — Camunda 7 с планом миграции.

ИСТОЧНИКИ

Migrate from Camunda 7 to
Camunda 8

camunda.com/platform-7/migrate/

Conceptual Differences |
Camunda 8 Docs

[docs.camunda.io – conceptual-
differences](https://docs.camunda.io/conceptual-differences)

Camunda 7 to Camunda 8
Migration Guide

[docs.camunda.io – migrating-from-
camunda-7](https://docs.camunda.io/migrating-from-camunda-7)